Literal Canonicalization in SKJS: Discharging Assumption **L-LIT** (Deep Dive G)

Models revision `cf5a6bc7` of `main` (2026-07-09).
Companion to *Refinement and Inference Growth in SKJS* (the main paper).

Abstract

The main soundness paper treats literal types abstractly through a denotation δ_k from stored literal spellings to runtime payloads, and records three claims about it as assumption **L-LIT**: (i) δ is a function, (ii) distinct (normalized) spellings denote $\approx$-inequivalent values, and (iii) the implementation's lexeme-level falsiness test agrees with the semantic definition of truthiness. For the numeric and bigint kinds, parts (ii) and (iii) **hold**: the parser normalizes literals to canonical form (`normalizeJSNumber`/`normalizeJSBigInt`), so the value 0 has exactly one spelling per kind. For the *string* kind they are VIOLATED (undecoded escapes). This deep dive gives the mathematics the canonicalization realizes and the string-kind gap. We define the denotation δ_k precisely for each literal kind, define a canonical form per kind (parse to the denoted value, re-render canonically, with -0 identified with 0 to match the main paper's $\approx$-closure semantics), and prove that over canonical forms **L-LIT**(i)–(iii) all hold, with the exact finite falsy canonical set per kind. We then describe where normalization lives, what it does to user-visible pretty-printing, and how it interacts with `PrimLiterals` set operations and numeric property names. Two adjacent items surface while proving the results: undecoded string escape sequences — a string-kind violation of **L-LIT**(ii), witnessed end-to-end (Finding 20) — and the ECMA magnitude bounds `isJSNumericString` enforces on exponent forms (Finding 21).

Contents

1 Purpose **2**

2 Recap of Definitions Used **3**

3 The Implementation Today, and the Gap **3**
3.1 Storage and comparison . . . 3
3.2 Construction sites of literal lexemes . . . 4
3.3 The falsiness test . . . 4
3.4 Witnessed violations . . . 4

4 Denotations δ_k **5**

5 Canonical Forms **6**
5.1 Numbers . . . 6
5.2 Bigints . . . 6
5.3 Strings, booleans, symbols . . . 7

6 Proofs over Canonical Forms **7**

7 Canonicalization **8**
7.1 Where normalization lives 8
7.2 Validation of over-lexed input 9
7.3 User-visible pretty-printing 9
7.4 Interaction with `PrimLiterals` operations 9
7.5 Numeric property names 9
7.6 Falsiness on canonical lexemes 9
7.7 Interaction with the test suites 10

8 Findings from Failed and Partial Proof Attempts **10**

9 Status and Registry **11**
9.1 L-LIT at the pin 11
9.2 Registry 11
9.3 Registry alignment 12
9.4 Pinned tests 12

1 Purpose

This paper serves the assumptions-and-obligations ledger of the main paper (its Section "Assumptions and Proof Obligations") as follows.

- **L-LIT(i)** (δ is a function): PROVED (definitional). This paper *strengthens* the row: we give δ_k concretely for every kind (Section 4), prove that no numeric literal denotes `NaN` (which the main paper's Definition of Literal Denotation assumes), and expose one caveat for kind `str` (escape decoding, Finding 20).

- **L-LIT(ii)** (injectivity modulo $\approx$): this paper defines the canonical forms and *proves* injectivity-modulo-$\approx$ over them (Theorem 14). For num/bigint it HOLDS: the parser normalizes literals (Section 7), modulo beyond-precision literals (below). For `str` it is VIOLATED (escape decoding, Finding 20).

- **L-LIT(iii)** (falsiness-exactness): this paper proves falsiness-exactness over canonical forms (Theorem 15), giving the exact finite falsy canonical set per kind. For num/bigint it HOLDS; for `str` it is VIOLATED (line-continuation falsiness), with an observable unsoundness witness (main paper, worked example "Known unsoundness: string-kind falsiness").

Downstream, the num/bigint discharge of (ii) and (iii) restores: the literal-filtering case of the main paper's Lemma "Residues are Sound" (the one soundness-relevant caveat in that document), and the exactness clause of its Theorem "Soundness of Constant Removal" (precision only); for `str` the caveat persists. No theorem statement or proof in the main paper depends on this; the string-kind violation is local to the leaf assumption, exactly as its summary asserts.

Numeric and bigint literals are normalized in the parser (`getNumber`/`getBigInt`, with `negateJSNumberLiteral` folding -0); Section 7 describes this and Section 9 the ledger row. The `str`-kind gap (Finding 20) is VIOLATED.

2 Recap of Definitions Used

We reuse the main paper's runtime model verbatim and re-state only what the proofs below manipulate; everything else is cited by name.

From the main paper, Section "Runtime Model":

- Values are tagged payloads, $\mathsf{Val} = \{(k, p) \mid k \in \mathsf{Tags},\ p \in \mathsf{Payload}_k\}$, with $\mathsf{Payload}_{\mathsf{num}} = \mathbb{F}$ (IEEE-754 doubles, including -0 and NaN), $\mathsf{Payload}_{\mathsf{str}} = \mathsf{String}$, $\mathsf{Payload}_{\mathsf{bigint}} = \mathbb{Z}$, $\mathsf{Payload}_{\mathsf{bool}} = \{\mathsf{true}, \mathsf{false}\}$, $\mathsf{Payload}_{\mathsf{sym}} = \mathsf{SymVal}$.
- Strict equality $\approx$: same tag and equal payloads, except that on numbers -0 is identified with 0 and NaN is equal to nothing, and the $\mathsf{undef}/\mathsf{void}$ tags are related cross-tag (irrelevant here: no literal type has a unit tag).
- Truthiness (main paper, Definition "Truthiness"): (num, n) is falsy iff $n \in \{0, -0, \mathsf{NaN}\}$; (str, s) falsy iff $s = \texttt{""}$; (bigint, z) falsy iff $z = 0$; (bool, b) falsy iff $b = \mathsf{false}$; symbols always truthy.
- Literal types: $\mathsf{prim}(k, L)$ with $L \subseteq \mathsf{Lit}_{\mathtt{k}}$ finite nonempty, and the *same-tag* $\approx$*-closure* semantics
$$[\![\mathsf{prim}(k, L)]\!] = \{\, v \mid \mathsf{rtag}(v) = \mathsf{tag}(k),\ \exists \ell \in L.\ v \approx (\mathsf{tag}(k), \delta_k(\ell)) \,\}$$
so the literal type `0` contains both 0 and -0, and $\mathsf{Lit}_{\mathtt{num}}$ is assumed to contain no NaN spelling.

For numbers we write $\doteq$ for the restriction of $\approx$ to num payloads: $x \doteq y$ iff x and y are equal as IEEE doubles with -0 identified with 0; $\mathsf{NaN} \not\doteq \mathsf{NaN}$. On $\mathbb{F} \setminus \{\mathsf{NaN}\}$, $\doteq$ is an equivalence whose only non-singleton class is $\{0, -0\}$.

Assumption L-LIT, as stated in the main paper's ledger. (i) δ is a function — definitional, used by the unconditional part (R2) of constant-removal soundness; (ii) *injectivity modulo* $\approx$ after normalization — distinct normalized lexemes denote $\approx$-inequivalent values; needed only for *exactness* claims; (iii) *falsiness-exactness* — the lexeme test `isFalsy` agrees with the Definition of Truthiness on every stored literal; needed by the residue lemma. Anchor: `PrimLiterals.sk`; `normalizeJSNumber`. Status: (i) PROVED, (ii)/(iii) VIOLATED by #655.

3 The Implementation Today, and the Gap

3.1 Storage and comparison

A literal is stored as a bare string with a position; identity, ordering, hashing, and all set operations are by that string: `PrimLiteral(pos, val)` projects to `val` and compares by it; `PrimLiterals` is a string-keyed array map whose `union/inter/diff/contains` are exact set operations *on lexemes*, and whose `xxhash` hashes only the strings (positions excluded).

impl: `PrimLiterals.sk / PrimLiteral, PrimLiterals (union, inter, diff, contains, xxhash)`

The kind (`num`, `bigint`, `str`, `bool`, `sym`) lives one level up, on `TPrimValue(kind, literals)`; `PrimLiteral` itself is kind-blind.

impl: `TType.sk / TPrimValue, TPrimValueKind`

3.2 Construction sites of literal lexemes

Every path that mints a numeric or bigint literal lexeme:

Site	Lexeme minted
`ParserUtils.sk / getNumber, getBigInt`	the normalized canonical form (`validate*` then `normalizeJSNumber`/`normalizeJSBigInt`)
`ParserType.sk / parsePrimary` (lines 69–70, 232–233)	type annotations `TANumLiteral`/`TABigIntLiteral`; negative forms prepend `"-"` to the raw lexeme
`TEnv.sk / normalize` (lines 1054, 1205)	converts those AST literals to `TPrimValue::single` unchanged
`TypeCheckExpr.sk / directLiteralSType`	expression literals (`Number`, `BigInt` tokens); unary `-` prepends `"-"`
`TypeCheckExpr.sk / asConstExpr` (`UOMinus` arm)	sign-flips by string prefix manipulation
`TEnv.sk` line 2162 (`keyof`)	string keys passing `isJSNumericString` become number literals with the same string
`SyntheticTuples.sk` line 127	tuple arity via `Int.toString()` (canonical decimal)
`TType.sk / truthinessValueResidue`	the constants `"0n"`, `""`

Every numeric and bigint site routes through the shared normalizer (`getNumber`/`getBigInt`), with the negative forms folded by `negateJSNumberLiteral`. Consequently the stored lexeme set for a source union `0x1 | 1` has *one* element, `"1"`, matching the semantic singleton.

3.3 The falsiness test

Truthiness classification of a literal is by lexeme: `TPrimValueKind::isFalsy` answers `literal == "false"` (bool), `literal == ""` (str), always-false (sym), and for num/bigint compares against the canonical zero: `literal == "NaN" || literal == "0"` (num) and `literal == "0n"` (bigint). This is *exact* because numeric and bigint literals are normalized to canonical form at parse time (`getNumber`/`getBigInt`, Section 7), so zero has exactly one spelling per kind: every zero spelling (`0.0`, `0e0`, `0_0`, `0x00`, `00`, and their bigint forms) has already been folded to `"0"`/`"0n"`. The num arm retains the `"NaN"` check — defensive only, since no construction site of Section 3.2 can mint it (Theorem 4 makes this precise).

impl: `TType.sk / TPrimValueKind::isFalsy (∼1348)`

Three consumers read this classification: `SType::falsiness` counts falsy literals per member (feeding the `safe`-mode condition guard in `TypeCheckUtils.sk / checkCondition`, which requires "boolean or a type with a single falsy value"); `SType::truthinessResidue` filters literals by `isFalsy` for `if`/`else` narrowing; and `truthinessValueResidue` does the same for the value of `&&`/`||`.

impl: `TType.sk / SType::falsiness, truthinessResidue, truthinessValueResidue; TypeCheckUtils.sk / checkCondition`

3.4 Witnessed violations

The live **L-LIT** violations are string-kind. Undecoded escape sequences break injectivity (ii) — two spellings of one string are stored distinct — and a line-continuation lexeme breaks falsiness (iii):

it denotes `""` yet is classified truthy. Both are witnessed end-to-end in Finding 20, and the main paper pins the falsiness case in its worked example "Known unsoundness: string-kind falsiness".

For the numeric and bigint kinds no such witness exists: parse-time normalization gives one spelling per value, so `falsiness` never double-counts a repeated zero and the stored lexeme set of a numeric union is injective over its classes (Theorems 15, 14). A union like `0.0 | 0x0` stores the single lexeme `"0"`; a condition `0 | ""` is rejected as a two-falsy-value condition and `0 | "a"` narrows correctly — all exactly per the model. The pinned test `acceptOctalZero` (`tests/projects/truthiness-safe/src/conditions.ts`, lines 106–115) exercises `0o0 | 1`, which normalizes to `0 | 1` and narrows soundly.

4 Denotations δ_k

The main paper assumes a denotation $\delta_k : \mathsf{Lit_k} \to \mathsf{Payload}_{\mathsf{tag}(k)}$. We now fix it per kind. $\mathsf{Lit_k}$ is the set of strings that can be stored for kind k by the construction sites of Section 3.2; for num/bigint these are ECMAScript *NumericLiteral*/*BigIntLiteral* lexemes optionally prefixed by `-` (the parser composes the sign), for str the unquoted token content, for bool {`"true"`, `"false"`}, and for sym synthesized identities.

Definition 1 ($\delta_{\texttt{num}}$)**.** *For a NumericLiteral body ℓ (after removing any leading `-` and all digit separators `_`): its* mathematical value $\mathrm{MV}(\ell) \in \mathbb{R}_{\geq 0}$ *is given by the ECMA-262 static semantics of NumericLiteral (decimal, or radix* 16/8/2 *under `0x`/`0o`/`0b` prefixes, exponents included), and $\delta_{num}(\ell)$ is* the Number value for $\mathrm{MV}(\ell)$*: the IEEE-754 round-to-nearest-ties-to-even image of that real in $\mathbb{F}$, with overflow to $+\infty$. A leading `-` negates the result (giving -0 when the body denotes 0). We also admit the two words `Infinity`/`-Infinity` with denotations $\pm\infty$ (they arise only as canonical renderings, never as source lexemes — `Infinity` lexes as an identifier).*

Definition 2 ($\delta_{\texttt{bigint}}$)**.** *For a BigIntLiteral $\ell = mn$ (optionally `-`-prefixed), $\delta_{bigint}(\ell) \in \mathbb{Z}$ is the signed integer value of m in its radix, separators removed. Exact; no rounding.*

Definition 3 ($\delta_{\texttt{str}}, \delta_{\texttt{bool}}, \delta_{\texttt{sym}}$)**.** *$\delta_{str}(\ell)$ is the string value ℓ denotes — i.e. the code-unit sequence* after decoding escape sequences *of the source string literal. δ_{bool} maps `"true"`/`"false"` to* true/false*. δ_{sym} maps each synthesized position-derived identity (`unique_symbol:file:sym:offset:name`) to a distinct element of* SymVal*; as the main paper notes, these are not source lexemes, so distinct stored strings have distinct denotations* by construction *— kind `sym` is already canonical and plays no further role here.*

impl: `TEnv.sk / uniqueSymbol`

Two remarks. First, $\delta_{\texttt{str}}$ as defined is the *semantic* denotation; the implementation stores the raw, *undecoded* content (Finding 20). Second, NaN is not in the range of $\delta_{\texttt{num}}$:

Theorem 4 (No NaN literal)**.** *For every $\ell \in \mathsf{Lit}_{num}$, $\delta_{num}(\ell) \neq \mathsf{NaN}$.*

Proof. $\mathrm{MV}(\ell)$ is a nonnegative real; the Number value of a real is a finite double or $\pm\infty$, never NaN, and negation preserves that. The only non-grammar lexemes admitted are `Infinity`/`-Infinity`, denoting $\pm\infty$. The remaining mint path, `keyof` over string keys, filters through `isJSNumericString`, whose grammar has no `NaN` production. □

This *proves* (rather than assumes) the main paper's stipulation that $\mathsf{Lit}_{\texttt{num}}$ contains no NaN spelling, and shows the `literal == "NaN"` arm of `isFalsy` is defensive: no construction site mints `NaN`.

impl: `JSNumericString.sk / isJSNumericString; TType.sk / isFalsy`

5 Canonical Forms

The canonicalization strategy is uniform: *parse the lexeme to the denoted value, then re-render that value by a fixed canonical rendering.* Injectivity then reduces to properties of the rendering, independent of how baroque the accepted input grammar is.

5.1 Numbers

The canonical rendering is JavaScript's own `Number :: toString` (radix 10), written $\mathsf{render}_{\mathtt{num}} : \mathbb{F} \setminus \{\mathsf{NaN}\} \to \mathsf{String}$. We treat IEEE-754 and the ECMA-262 rendering algorithm as an abstract interface with three axioms (the "abstracted-with-axioms" bin of the modeling contract); each is a stated property of ECMA-262 §6.1.6.1.20 (*Number::toString*).

Axiom 5 (NUM-FUN)**.** render_{num} *is a total function on* $\mathbb{F} \setminus \{\mathsf{NaN}\}$*, with* $\mathsf{render}_{num}(\pm\infty) =$ *`"Infinity"`*/*`"-Infinity"`*.

Axiom 6 (NUM-RT (round-trip))**.** *For every finite* x*,* $\delta_{num}(\mathsf{render}_{num}(x)) \doteq x$*. (ECMA-262 chooses digits* s *and exponent* n *such that* the Number value for $s \cdot 10^{n-k}$ is x*; parsing the rendered string recovers exactly that value. For* $x \doteq 0$ *the rendering is `"0"`, which parses to* $+0 \doteq x$*.) For* $x = \pm\infty$ *the round-trip is by the `Infinity` clause of Definition 1.*

Axiom 7 (NUM-ZID (zero identification))**.** $x \doteq y \implies \mathsf{render}_{num}(x) = \mathsf{render}_{num}(y)$*. (The only non-singleton* $\doteq$*-class is* $\{0, -0\}$*, and the algorithm returns `"0"` for both — the sign of* -0 *is dropped.)*

Definition 8 (Canonical numeric lexemes; $\mathsf{canon}_{\mathtt{num}}$)**.** $\mathsf{Canon}_{num} \triangleq \mathsf{render}_{num}(\mathbb{F} \setminus \{\mathsf{NaN}\})$*, and* $\mathsf{canon}_{num} \triangleq \mathsf{render}_{num} \circ \delta_{num} : \mathsf{Lit}_{num} \to \mathsf{Canon}_{num}$.

Justification of the rendering choice. (1) $-0 \mapsto$ `"0"` (Axiom NUM-ZID) implements the main paper's $\approx$-closure semantics of literal types — the literal type `0` already denotes $\{0, -0\}$, so a distinct `-0` canonical spelling would be a second name for the same type; tsc likewise gives `const x = -0` the type `0`. (2) $\mathsf{NaN} \notin \mathrm{dom}(\mathsf{render}_{\mathtt{num}})$ matches the main paper's exclusion of a NaN literal type (its denotation would be $\emptyset$ under closure semantics), and Theorem 4 shows totality is not thereby lost. (3) The finite part of $\mathsf{Canon}_{\mathtt{num}}$ is exactly the strings satisfying `String(Number(s))` $= s$, the predicate the codebase already uses to classify numeric property names — so `keyof`-minted number literals and canonicalized literals agree by construction (Finding 21 details how the predicate enforces the magnitude bounds). (4) It is what tsc renders in hovers and errors, minimizing user-visible divergence.

impl: `JSNumericString.sk / isJSNumericString (intended grammar)`

Note $\mathsf{Canon}_{\mathtt{num}}$ contains renderings such as `"1e-7"` whose signed exponent the SKJS lexer cannot tokenize as a single numeric token (Section 7.2); this is harmless because stored lexemes are never re-lexed, and tsc accepts these spellings.

5.2 Bigints

Here everything is concrete; no axioms are needed.

Definition 9 (Canonical bigint lexemes)**.** $\mathsf{render}_{\mathbb{Z}}(z)$ *is the standard signed decimal numeral of* $z \in \mathbb{Z}$ *(no leading zeros, no separators, no radix prefix, no sign for* $z \geq 0$*, so* $\mathsf{render}_{\mathbb{Z}}(0) =$ *`"0"` — in particular `-0n` normalizes to `0n`), and* $\mathsf{Canon}_{bigint} \triangleq \{\mathsf{render}_{\mathbb{Z}}(z)\,\mathtt{n} \mid z \in \mathbb{Z}\}$*, with* $\mathsf{canon}_{bigint} = (\mathsf{render}_{\mathbb{Z}} \circ \delta_{bigint})\,\mathtt{n}$.

Fact 10. $\mathsf{render}_{\mathbb{Z}}$ *is a bijection* $\mathbb{Z} \to \mathsf{render}_{\mathbb{Z}}(\mathbb{Z})$*, and* $\delta_{\texttt{bigint}}(\mathsf{render}_{\mathbb{Z}}(z)\,\texttt{n}) = z$*. (Uniqueness of positional decimal numerals without leading zeros; elementary.)*

5.3 Strings, booleans, symbols

$\mathsf{Canon}_{\texttt{str}} \triangleq \mathsf{String}$: the canonical form of a string literal is *the denoted string itself* ($\mathsf{canon}_{\texttt{str}} = \delta_{\texttt{str}}$, i.e. decode escapes at the quote boundary and store code units). $\mathsf{Canon}_{\texttt{bool}} = \{\texttt{"true"}, \texttt{"false"}\}$ and $\mathsf{Canon}_{\texttt{sym}}$ = the synthesized identities: already canonical, canon is the identity on them (Definition 3).

Definition 11 (Canonical-storage invariant)**.** *A* `TPrimValue(kind k, Some(lits))` *is* canonical *when every stored lexeme is in* Canon_k*. Parse-time normalization (Section 7) establishes this for num/bigint, except* beyond-precision *literals (radix* $> 2^{53}$*, out-of-range exponents), which are stored in spelled form and kept sound by never merging (Section 7).*

6 Proofs over Canonical Forms

Throughout this section literals are canonical (Definition 11); kind `sym` is covered by construction and omitted from the case analyses where it is trivial.

Lemma 12 (Idempotence)**.** *For* $k \in \{\texttt{num}, \texttt{bigint}\}$ *and* $\ell \in \mathsf{Canon}_k$*:* $\mathsf{canon}_k(\ell) = \ell$*.*

Proof. `num`: $\ell = \mathsf{render}(x)$ for some x; then $\mathsf{canon}(\ell) = \mathsf{render}(\delta(\mathsf{render}(x)))$, and by NUM-RT $\delta(\mathsf{render}(x)) \doteq x$, so by NUM-ZID $\mathsf{render}(\delta(\mathsf{render}(x))) = \mathsf{render}(x) = \ell$ (infinite x: direct). `bigint`: immediate from Fact 10. □

Theorem 13 (L-LIT(i): δ is a function; totality)**.** *Each* δ_k *is a well-defined function on* Lit_k*, and each* canon_k *is a well-defined function* $\mathsf{Lit}_k \to \mathsf{Canon}_k$*. For* `str` *this holds with* $\delta_{\texttt{str}}$ *as specified in Definition 3; the implementation realizes a different function for* `str` *(raw storage), see Finding 20.*

Proof. `num`: MV is defined by a deterministic grammar attribution; rounding is a function; Axiom NUM-FUN gives the rendering (Theorem 4 keeps NaN out of its domain). `bigint`: radix valuation and Fact 10. `str`: escape decoding is deterministic. `bool`, `sym`: finite/by-construction tables. □

Theorem 14 (L-LIT(ii): injectivity modulo $\approx$)**.** *For every kind* k *and* $\ell_1, \ell_2 \in \mathsf{Canon}_k$*: if* $(\mathsf{tag}(k), \delta_k(\ell_1)) \approx (\mathsf{tag}(k), \delta_k(\ell_2))$ *then* $\ell_1 = \ell_2$*. Equivalently: distinct canonical lexemes denote* $\approx$*-inequivalent values, so each* $\approx$*-class of payloads meets at most one canonical lexeme.*

Proof. `num`: on num payloads $\approx$ is $\doteq$. Write $\ell_i = \mathsf{render}(x_i)$. By NUM-RT $\delta(\ell_i) \doteq x_i$ (for infinite x_i, $\delta(\ell_i) = x_i$), so $\delta(\ell_1) \doteq \delta(\ell_2)$ gives $x_1 \doteq x_2$ by symmetry/transitivity of $\doteq$ on its domain (NaN is not in the range of δ by Theorem 4). NUM-ZID then gives $\ell_1 = \mathsf{render}(x_1) = \mathsf{render}(x_2) = \ell_2$. `bigint`: $\approx$ is equality on $\mathbb{Z}$; Fact 10 (bijectivity) concludes. `str`: $\mathsf{canon}_{\texttt{str}}$ stores the value itself, so δ restricted to $\mathsf{Canon}_{\texttt{str}}$ is the identity — injective. `bool`: two-point table, injective. `sym`: by construction (main paper, Definition "Literal Denotation"). □

Theorem 15 (L-LIT(iii): falsiness-exactness)**.** *Define the* falsy canonical sets

$$F_{\texttt{bool}} = \{\texttt{"false"}\}, \quad F_{\texttt{num}} = \{\texttt{"0"}\}, \quad F_{\texttt{str}} = \{\texttt{""}\}, \quad F_{\texttt{bigint}} = \{\texttt{"0n"}\}, \quad F_{\texttt{sym}} = \emptyset.$$

For every kind k *and* $\ell \in \mathsf{Canon}_k$*:* $\mathsf{truthy}(\mathsf{tag}(k), \delta_k(\ell)) = \mathsf{false} \iff \ell \in F_k$*. Hence over canonical forms the lexeme test "*$\ell \in F_k$*" agrees exactly with the main paper's Definition of Truthiness, and each* F_k *is finite (at most one element).*

Proof. By the Definition of Truthiness, the falsy payloads are: false (bool); $\{0, -0, \mathsf{NaN}\}$ (num); `""` (str); 0 (bigint); none (sym). `num`: $\mathsf{NaN} \notin \mathrm{range}(\delta)$ (Theorem 4); $\delta(\ell) \in \{0, -0\}$ iff $\delta(\ell) \doteq 0$ iff (by Lemma 12 and NUM-ZID) $\ell = \mathsf{canon}(\ell) = \mathsf{render}(\delta(\ell)) = \mathsf{render}(0) =$ `"0"`; conversely $\delta($`"0"`$) = 0$ is falsy. `bigint`: $\delta(\ell) = 0$ iff $\ell =$ `"0n"` by Fact 10. `str`: identity denotation, falsy iff the stored string is empty. `bool`, `sym`: by the tables. □

Corollary 16 (Residue lemma discharged)**.** *With canonical storage and* ***`isFalsy`*** *replaced by membership in F_k (a single string equality per kind), the literal-filtering case of the main paper's Lemma "Residues are Sound" holds unconditionally: the kept-literal sets of tres_b and vres_b contain exactly the literals whose denotations have truthiness b.*

impl: `TType.sk / truthinessResidue, truthinessValueResidue`

Corollary 17 (Exactness of constant removal)**.** *The exactness clause of the main paper's Theorem "Soundness of Constant Removal" holds under canonical storage: its stated side condition is precisely L-LIT(ii), Theorem 14. (Its soundness clauses R1/R2 were unconditional already.)*

impl: `TType.sk / removeConstant; PrimLiterals.sk / diff`

Corollary 18 (Lexeme set operations compute value set operations)**.** *For canonical $L_1, L_2 \subseteq \mathsf{Canon}_k$, writing $[\![L]\!] \triangleq [\![\mathsf{prim}(k, L)]\!]$: $[\![L_1 \cup L_2]\!] = [\![L_1]\!] \cup [\![L_2]\!]$ (unconditional); $[\![L_1 \cap L_2]\!] = [\![L_1]\!] \cap [\![L_2]\!]$; and $[\![L_1 \setminus L_2]\!] = [\![L_1]\!] \setminus [\![L_2]\!]$.*

Proof. Union: $[\![\cdot]\!]$ is a union of $\approx$-closures, which commutes with $\cup$. Intersection and difference: by Theorem 14 the $\approx$-classes named by distinct canonical lexemes are disjoint, so a value $v \approx \delta(\ell_1)$ with $\ell_1 \in L_1$ lies in $[\![L_2]\!]$ iff $\ell_1 \in L_2$; the class-wise identities follow. (Without canonicity only $\subseteq$ / $\supseteq$ over-approximations hold, which is why `PrimLiterals::inter` can spuriously produce $\bot$ today: $\{$`"0"`$\} \cap \{$`"0x0"`$\} = \emptyset$ on lexemes while the value sets coincide — an unsound *emptiness* verdict wherever $\bot$ is read as disjointness.)

impl: `PrimLiterals.sk / union, inter, diff` □

Corollary 19 (Exact falsy counting)**.** *Over canonical storage,* ***`SType::falsiness`*** *counts, per literal member, the exact number of falsy $\approx$-classes (0 or 1, since $|F_k| \leq 1$), so* ***`FalsyCount`*** *never double-counts a class across spellings (the `0 | 0x0` witness of Section 3.4 disappears) and the* ***`safe`*** *mode condition guard implements its intended semantics on literal unions.*

impl: `TType.sk / SType::falsiness, FalsyCount; TypeCheckUtils.sk / checkCondition`

7 Canonicalization

7.1 Where normalization lives

Normalization lives at one choke point, the parser. `getNumber`/`getBigInt` (`ParserUtils.sk:20-42`) call `validateJSNumericLiteral`/`validateJSBigIntLiteral` then `normalizeJSNumber`/`normalizeJSBigInt`. Two consequences of parser placement: (1) the same helpers feed numeric *property names* (`ParserDefs.sk / getNameOrStringOrNumber`), so property names canonicalize too; (2) the negative sites, which compose `"-"` after the helper returns (`ParserType.sk:233,235`; `directLiteralSType`; `asConstExpr`), route through `negateJSNumberLiteral` (`JSNumericString.sk:479`), which folds $-0 \to 0$ and $-0n \to 0n$ over the normalized lexeme. Every num/bigint `PrimLiteral` therefore carries a canonical lexeme by construction, establishing Definition 11 (modulo beyond-precision literals). Boolean and symbol constructors are already canonical, as are the synthesized constants (`SyntheticTuples.sk` 127, `truthinessValueResidue`'s `"0n"`/`""`).

impl: `ParserUtils.sk / getNumber, getBigInt; JSNumericString.sk / normalizeJSNumber, normalizeJSBigInt, negateJSNumberLiteral`

7.2 Validation of over-lexed input

The numeric-token loop in the lexer accepts a strict superset of the NumericLiteral grammar — any run over {`0-9`, `a-f`, `A-F`, `e`, `E`, `.`, `x`, `X`, `b`, `B`, `o`, `O`, `_`} — so lexemes like `1.2.3`, `0x`, or `1__2` reach `getNumber`. `validateJSNumericLiteral`/`validateJSBigIntLiteral` reject these non-grammar lexemes with a diagnostic before `normalizeJSNumber`/`normalizeJSBigInt` runs. Signed exponents (`1e-5`) never arrive as one token (the loop stops at `-`/`+`), so the normalizer's input grammar has unsigned exponents only, while its *output* may contain signed ones (`"1e-7"`), which the lexer does not re-tokenize as a single numeric token.

impl: `Lexer.sk / numeric-token loop (∼384--412); ParserUtils.sk / getNumber, getBigInt`

7.3 User-visible pretty-printing

Literal types print their stored lexemes (`PrimLiterals::toStringList`), so diagnostics and hovers show the canonical spelling: `0o0 | 1` prints as `0 | 1`, `0x1 | 1` collapses to `1`. This matches tsc's rendering. Two corners: (1) large/small magnitudes print in canonical scientific form (`1e+21`, `1e-7`) — valid TS literal spellings, though SKJS's own lexer cannot re-lex the signed-exponent ones (§7.2); (2) overflowed literals print as `Infinity`, which is *not* a round-trippable literal-type spelling.

impl: `PrimLiterals.sk / toStringList; TTypePp.sk line 210`

7.4 Interaction with `PrimLiterals` operations

`union`/`inter`/`diff`/`contains`/`xxhash` are lexeme-level; under the canonical-storage invariant they are exact value-set operations (Corollary 18). In particular `sameConstant` and structural type hashing are value-respecting, and `TFields::isTuple`'s `toIntOption` on the `length` literal resolves radix-spelled lengths through their canonical form (canonical `"2"` succeeds where a raw `"0x2"` would fail `toIntOption`).

impl: `PrimLiterals.sk; TType.sk / TPrim::sameConstant, TFields::isTuple (∼2105)`

7.5 Numeric property names

Numeric *property names* take the same normalization path (`ParserDefs.sk/getNameOrStringOrNumber`, lines 64–71) and end up as field-name strings compared against literal-type lexemes (`FieldKey::ofSType`; `TFields::maybeGet`; `namedFieldsForKeyKind` filters names by `isJSNumberKeyName`). TS defines a numeric property name as `String(Number(name))`, i.e. exactly $\mathsf{Canon}_{\mathsf{num}}$'s finite part. For coherence, a declaration `{ 0x10: T }` must store the field name `"16"` so that the canonicalized access key `obj[0x10]` (literal type `16`) resolves to it — otherwise declaration-side and use-side spellings would sit permanently disconnected. Parser-side normalization of the declaration name is what keeps the two sides aligned.

impl: `ParserDefs.sk / getNameOrStringOrNumber; TType.sk / FieldKey::ofSType, namedFieldsForKeyKind`

7.6 Falsiness on canonical lexemes

Because num/bigint lexemes are canonical, the falsiness test reads them directly. `isFalsy` (`TType.sk:1348`) compares against the canonical falsy sets of Theorem 15: `TNum() -> literal == "NaN" || literal == "0"` (the `"NaN"` arm is defensive, though Theorem 4 shows no site mints it); `TBigInt() -> literal == "0n"`; other kinds compare against their own canonical falsy lexemes. There is no separate zero-spelling predicate: a single canonical string per value makes the equality test exact.

7.7 Interaction with the test suites

The test suites are `@skjs-expect-error` annotation-based with no expected-output snapshots, so no golden files capture literal-type rendering; error-message text carries canonical spellings. `acceptOctalZero` (`tests/projects/truthiness-safe/src/conditions.ts` 106–115) annotates with `0o0`: both the union member and the else-branch annotation `const e: 0o0` normalize to `0`, so the assignment checks against the canonical form.

8 Findings from Failed and Partial Proof Attempts

Per the series contract, unprovable steps are reported as findings, not silently narrowed away.

Finding 20 (String escapes: L-LIT(ii) fails for kind `str`, witnessed)**.** *Attempting to prove Theorem 14 for `str`* as implemented *fails. The lexer skips escape sequences without decoding them (`Lexer.sk` string loops, ∼239–277: on \ it advances two positions), and `unquoteString` only strips the quotes, with an explicit `TODO: handle escaped sequences`. So the literal types written `"\x61"` and `"a"` store* distinct *lexemes denoting the* same *runtime string — a second violation of L-LIT(ii), independent of #655 and not covered by its fix. Consequences mirror the numeric case: `inter` can return ⊥ for overlapping string literal types (unsound where ⊥ is read as disjointness), `diff`/exactness degrade, and equality narrowing against an escape-spelled constant misclassifies. Falsiness is* also *affected, contrary to an earlier belief that no non-empty lexeme can denote `""`: ECMA-262* LineContinuation *(backslash + line terminator) has string value `""`, so the non-empty backslash-newline lexeme denotes the falsy `""` while `isFalsy` (`lexeme == ""`) reads it truthy. This is a string-kind violation of **L-LIT**(iii), witnessed end-to-end: a union `"..." | 0` whose string member is a line-continuation lexeme is accepted with `const z: 0 = x` in the `else` of `if (x)` while that branch runs at `x = ""` (the canonical `"" | 0` control is correctly rejected). The injectivity failure is witnessed too: with `x: "a" | 0`, the test `x === "\x61"` types its (runtime-taken) true branch `never`, and `const z: 0 = x` is accepted while `z` holds the string `"a"`; the checker even emits an always-false warning for a comparison that is true at runtime. Issue draft: `issues/string-literal-escapes-undecoded.md`. Absent decoding, Theorem 14's `str` case holds only under the assumption "stored lexeme = denoted string," which is exactly* canon_{str} *of §5.3.*

impl: `ParserUtils.sk / unquoteString; Lexer.sk / string loops`

Finding 21 (`isJSNumericString` enforces the ECMA-262 magnitude bounds)**.** *The finite part of* Canon_{num} *(Definition 8) is the round-trip characterization* $\mathtt{String}(\mathtt{Number}(\mathtt{s})) = s$, *and `isJSNumericString` realizes it. Its scientific production* $[-][1\text{-}9](.d^*)\mathtt{e}[\pm]e^+$ *carries ECMA-262's magnitude side condition (`JSNumericString.sk:208`): scientific rendering is canonical only when the printed exponent is* ≥ 21 *or* ≤ -7, *so a form such as `"1e+5"` — whose value renders as* $\mathtt{String}(\mathtt{Number}(\text{"1e+5"})) = \texttt{"100000"}$ *— is rejected rather than admitted as a second spelling of the same value. Injectivity (Theorem 14) thus holds on the predicate's language. A separate `isJSNumberKeyName` predicate (`:126`) serves the index-signature clients, so the `keyof` and number-index paths do not mint non-canonical number literals from string keys.* Canon_{num} *in this paper is defined by the rendering, not by the predicate, so the theorems are stated independently of the predicate's exact grammar; adversarial probing of the `keyof` and number-index paths found* no soundness leak.

impl: `JSNumericString.sk / isJSNumericString (bounds :208), isJSNumericExponentPartIter (:184), isJSNumberKeyName (:126); TEnv.sk:2255; TType.sk:1927`

Finding 22 (Why normalization is at construction, not at use sites)**.** *The alternative — keep raw lexemes and parse to values inside* `isFalsy`*,* `inter`*,* `diff`*,* `sameConstant`*,* `xxhash`*, subtyping, and printing — is inferior. The comparison surface is wide and grows (any consumer of* `PrimLiteral.val` *reading the raw lexeme would revert to lexeme semantics), the hash requires a canonical key anyway (hashing raw spellings would break hash-consistency of semantically equal types), and per-use parsing would re-do work the constructor does once. Normalize-at-construction keeps the invariant local and makes every downstream operation correct by Corollary 18 without per-site logic. This mirrors the issue tracker's own recommendation on #655.*

9 Status and Registry

9.1 L-LIT at the pin

Item	At the pin
L-LIT(i)	holds (δ_k concrete per kind, §4; NaN-freeness of $\mathsf{Lit}_{\mathtt{num}}$ a theorem, Thm. 4); the `str` raw-storage caveat (Finding 20) stands absent escape decoding
L-LIT(ii)	holds for num/bigint/bool/sym (injectivity over canonical forms, Thm. 14); for `str` it fails, witnessed end-to-end (Finding 20, issue draft `issues/string-literal-escapes-undecoded.md`)
L-LIT(iii)	holds for num/bigint/bool/sym; for `str` it fails (Finding 20 witnesses both an (ii) and an (iii) string-kind violation)
Residue lemma (main paper)	unconditional (Cor. 16)
rmc exactness (main paper)	unconditional (Cor. 17)
Storage	parse-time `normalizeJSNumber`/`normalizeJSBigInt` (= canon_k of §5); property names normalize via the shared `getNumber` (§7.5); the `isJSNumericString` thresholds (Finding 21) are enforced. Carve-out: beyond-precision literals ($> 2^{53}$, out-of-range exponents) are left non-canonical — kept sound by never merging — so the canonical-storage invariant of Definition 11 is only *partial* for `num`

For num/bigint, L-LIT(ii)/(iii) hold under the canonical-storage invariant; the `str`-kind violation is witnessed (Finding 20).

9.2 Registry

The authoritative, current statuses are maintained in the registry (`docs/ledger.yaml`), rendered as the main paper's Assumptions and Proof Obligations tables; this paper is cited there as evidence. The registry rows it supports:

Row	Statement	Status
L-LIT	Literal spellings: (i) δ is a function; (ii) injectivity modulo $\approx$ after normalization; (iii) falsiness-exactness of the lexeme test.	VIOLATED

Imports (review contract). The named statements this paper assumes from elsewhere. A reviewer treats each as an axiom, checking citations against the registry's canonical statement; no other paper's internals are in scope:
None beyond the main paper's definitions.

9.3 Registry alignment

The registry records, consistent with this paper:

1. **L-LIT status.** (i) PROVED; (ii)/(iii) PROVED (this paper, Thms. 14/15) for num/bigint/bool/sym under the canonical-storage invariant, pinned by the tests below; for `str`, (ii)/(iii) VIOLATED (its own issue, escape decoding). The row anchors `normalizeJSNumber`/`normalizeJSBigInt`.

2. **Main paper, worked example "Known unsoundness: string-kind falsiness".** The remaining witnessed L-LIT unsoundness is the string kind; the generated summary's VIOLATED grouping carries its string-kind entry.

3. **Main paper, Definition "Literal Denotation".** The premise "$\mathsf{Lit}_{\mathtt{num}}$ contains no NaN spelling" is a citation of Theorem 4.

9.4 Pinned tests

In `tests/projects/truthiness-safe/src/conditions.ts` unless noted:

1. `x: 0.0 | "" — if (x)` is `@skjs-expect-error` (two falsy values), matching the `0 | ""` control, since both spellings of zero canonicalize to `0`.

2. `x: 0.0 | "a" — if (x)` accepted; then-branch narrows to `"a"`, else-branch to `0` (and `const e: 0.0 = x` checks, both spellings canonicalizing).

3. `x: 0 | 0x0` — accepted as a condition (the union collapses to one falsy class); pins Corollary 19.

4. `acceptOctalZero`, whose `0o0` annotation canonicalizes to `0`.

5. A rendering pin: a diagnostic that prints a literal type declared `0o0 | 1` shows `0 | 1`.

6. (With Finding 21) `keyof` over `{ "1e+5": T }` does *not* yield a numeric literal member.

7. (With Finding 20, separate issue) an equality or assignment mixing `"a"` and `"\x61"` literal types, pinned to the string-kind behavior of Finding 20.

Anchor cross-check hooks per the main paper's maintenance appendix: the L-LIT row anchors `normalizeJSNumber`/`normalizeJSBigInt` and `unquoteString` (for the string-kind gap).